

경북대학교 공학석사학위논문

강화학습을 이용한 eBPF 커널 신호 기반 MQTT 꼬리 지연 제어에 관한 연구

대학원 컴퓨터학부

정진욱

2025년 9월

경북대학교 대학원

강화학습을 이용한 eBPF 커널 실행
MOTI 포리 지연 제어에 관한 연구 기반

(공학
위학
논석
무사
)

정

진

욱

2
0
2
5

강화학습을 이용한 eBPF 커널 신호 기반 MQTT 꼬리 지연 제어에 관한 연구

이 논문을 공학석사 학위논문으로 제출함

대학원 컴퓨터학부

정진욱

지도교수 권영우

정진욱의 공학석사 학위논문을 인준함

2025년 9월

위원장 _____ (인)

_____ (인)

_____ (인)

경북대학교 대학원위원회

강화학습을 이용한 eBPF 커널 신호 기반 MQTT 꼬리 지연 제어에 관한 연구

Jinuk Jung

Department of Computer Science and Engineering

Graduate School, Kyungpook National University

Daegu, Korea

(Supervised by Professor Young-woo Kwon)

(Abstract) 사물 인터넷(IoT) 기술이 스마트 팩토리, 커넥티드 카 등 다양한 산업 분야로 확산됨에 따라, 수많은 장치로부터 발생하는 데이터를 효율적으로 처리하기 위한 경량 메시징 프로토콜의 중요성이 커지고 있다. MQTT(Message Queuing Telemetry Transport)는 발행/구독 모델과 낮은 오버헤드를 바탕으로 IoT 환경의 사실상 표준 프로토콜로 자리 잡았다. 그러나 수천 개의 장치가 동시에 높은 빈도로 메시지를 발행하는 고밀도 트래픽 환경에서는 네트워크 혼잡으로 인해 일부 메시지의 지연

시간이 수 초 단위로 급증하는 '꼬리 지연 시간(Tail Latency)' 문제가 발생한다. 이러한 극단적인 지연 시간은 실시간 제어 시스템의 신뢰성을 저해하는 심각한 요인이다.

기존의 꼬리 지연 시간 완화 기법들은 클라이언트 로직 수정, 브로커 내부 구조 변경, 또는 전송 프로토콜 교체 등을 요구하여 높은 통합 비용과 운영 복잡성을 야기한다. 본 논문에서는 이러한 한계를 극복하기 위해, 애플리케이션 및 브로커 수정 없이 엡지 게이트웨이에 투명하게 배포 가능한 지능형 제어 시스템 'eMQTT-RL'을 제안한다. eMQTT-RL은 리눅스 커널의 eBPF(extended Berkeley Packet Filter) 기술을 활용하여 TCP 스택의 핵심 성능 지표(RTT, 재전송, 송수신 버퍼 점유율)를 실시간으로 관측한다.

본 연구의 핵심은 기존의 정적 임계값 기반 휴리스틱 제어 방식을 넘어, 심층 강화학습(Deep Reinforcement Learning, DRL)을 통해 동적이고 예측적인 제어 정책을 자율적으로 학습하는 에이전트를 설계한 점에 있다. 제어 문제를 마르코프 결정 과정(Markov Decision Process, MDP)으로 정형화하고, Proximal Policy Optimization (PPO) 알고리즘을 적용하여 eBPF로 관측된 커널 신호를 상태(State)로 입력받아 MQTT 발행자의 전송률(Send Rate)과 배치 크기(Batch Size)를 최적으로 조절하는 행동(Action)을 학습한다. 보상 함수(Reward Function)는 처리량(Throughput)을 유지하면서 꼬리 지연 시간(p99 Latency)에 강력한 페널티를 부과하도록 설계하여, 시스템이 지연 시간 안정화에 집중하도록 유도한다.

20개의 발행자가 초당 총 1,000개의 메시지를 생성하는 고밀도 트래픽 시나리오에서 eMQTT-RL의 성능을 평가한 결과, 제안 시스템은 기존 휴리스틱 제어 방식(eMQTT-AC) 및 제어 없는 기저 상태(Baseline) 대비

월등한 성능을 보였다. 특히 99분위(p99) 종단 간 지연 시간을 기저 상태의 2,088.54 ms에서 251.72 ms로 약 88% 감소시켰으며, 이는 휴리스틱 방식의 473.50 ms보다도 46.8% 더 개선된 수치이다. 이러한 지연 시간 안정화는 처리량 손실 없이 달성되었으며, CPU 오버헤드는 2-3% 수준으로 엣지 게이트웨이에 적용 가능한 경량성을 입증하였다. 본 연구는 eBPF의 커널 관측성과 강화학습의 지능적 의사결정 능력을 결합하여, 복잡한 IoT 엣지 환경에서 MQTT 통신의 신뢰성과 실시간성을 보장하는 새로운 패러다임을 제시한다.

1 서론

1.1 연구 배경: IoT 엣지 컴퓨팅과 MQTT의 부상

지난 10년간 사물 인터넷(IoT)은 스마트 팩토리, 자율 주행, 스마트 시티 등 산업 전반에 걸쳐 혁신을 주도하는 핵심 기술로 자리매김했다. 이러한 시스템들은 수천에서 수만 개에 이르는 센서와 액추에이터 장치들이 생성하는 방대한 양의 원격 측정(telemetry) 데이터를 수집하고 전파하는 능력에 의존한다. 데이터 처리의 패러다임은 중앙 집중형 클라우드에서 데이터가 생성되는 위치와 가까운 곳에서 처리하는 엣지 컴퓨팅(Edge Computing)으로 전환되고 있다. 엣지 컴퓨팅은 데이터 전송 지연 시간을 줄이고, 네트워크 대역폭 사용을 최적화하며, 데이터 프라이버시를 강화하는 등 다양한 이점을 제공한다.

이러한 분산된 엣지 환경에서 장치 간의 효율적인 통신을 위해서는 경량 메시징 프로토콜이 필수적이다. 그중에서도 MQTT(Message Queuing Telemetry Transport)는 발행/구독(Publish/Subscribe) 모델을 기반으로 한 간결한 프로토콜 구조와 낮은 오버헤드 덕분에, 컴퓨팅 성능과 전력이 제한적인 IoT 장치 환경에서 사실상의 표준으로 채택되었다. MQTT는 중앙의 브로커(Broker)를 통해 메시지를 중개함으로써 송신자와 수신자를 분리(decouple)하여, 시스템의 확장성과 유연성을 크게 향상시킨다. 엣지 게이트웨이는 로컬 운영 기술(OT) 네트워크와 클라우드를 연결하는 중요한 지점에 위치하며, MQTT

브로커의 역할을 수행하는 경우가 많다. 이로 인해 엣지 게이트웨이는 수많은 장치로부터 트래픽이 집중되는 잠재적인 병목 지점이 된다.

1.2 문제 정의: MQTT 꼬리 지연 시간

현대의 IoT 애플리케이션, 특히 스마트 팩토리의 로봇 제어나 커넥티드 카의 안전 시스템과 같이 실시간성이 중요한 분야에서는 평균적인 응답 시간뿐만 아니라 최악의 경우에 대한 응답 시간 보장이 매우 중요하다. 시스템의 전체 성능은 가장 느린 구성 요소에 의해 결정되는 경우가 많기 때문이다. 여기서 '꼬리 지연 시간(Tail Latency)'이라는 문제가 부상한다. 꼬리 지연 시간이란, 전체 요청 중 상위 95분위(p95) 또는 99분위(p99)에 해당하는 극단적인 지연 시간을 의미한다. 평균 지연 시간이 양호하더라도, 일부 메시지의 지연 시간이 수 초 단위로 급증하게 되면 실시간 제어 루프의 안정성을 해치고 전체 시스템의 신뢰도를 심각하게 저하시킬 수 있다.

고밀도, 고부하 트래픽 환경은 꼬리 지연 시간 문제를 더욱 악화시킨다. 수많은 장치가 동시에 높은 빈도로 메시지를 발행하면, 발행자 측의 커널 송신 버퍼(sk_wmem_queued), 네트워크 장비, 그리고 브로커 내부의 수신/토픽/세션 큐 등 시스템 경로 전반에 걸쳐 큐가 쌓이게 된다. 이러한 큐잉(queueing) 현상은 지연 시간을 증폭시키는 주된 원인이다. 특히 엣지 환경은 제한된 컴퓨팅 자원, 클라우드로 향하는 비대칭적인 업링크 대역폭, 유선, Wi-Fi, 셀룰러 등 이기종의 마지막 홉(last-hop) 링크와 같은 제약 조건들로 인해 이 문제에 더욱 취약하다. 따라서 엣지 환경에 적용

가능한 꼬리 지연 시간 해결책은 다음과 같은 엄격한 요구사항을 만족해야 한다: (i) 애플리케이션 투명성, (ii) 브로커 비종속성, (iii) 범용 엣지 게이트웨이에 적용 가능한 경량성, (iv) 기존 트래픽 중단 없는 점진적 배포 가능성.

1.3 기존 접근법의 한계

MQTT 꼬리 지연 시간을 완화하기 위해 다양한 접근법이 시도되었으나, 각각 명확한 한계를 가지고 있다.

첫째, 클라이언트 측에서 재시도(retry)나 헤징(hedging, 중복 요청 전송) 로직을 추가하는 방식은 모든 장치에 변경 사항을 배포해야 하므로 관리 복잡성과 비용이 크다. 또한, 네트워크가 불안정한 상황에서 모든 클라이언트가 동시에 재시도를 수행하면 오히려 혼잡을 가중시키는 '재시도 폭풍(retry storm)'을 유발할 수 있다.

둘째, 브로커 내부의 큐 관리 정책이나 스케줄링 알고리즘을 개선하는 방식은 특정 브로커 구현에 종속적이며, 이미 운영 중인 시스템에 적용하기 어렵다는 단점이 있다. 또한, 브로커 수준의 변경은 전문적인 지식을 요구하며, 잘못된 튜닝은 오히려 성능을 저하시킬 수 있다.

셋째, TCP를 QUIC과 같은 최신 전송 프로토콜로 대체하는 방안은 전송 계층의 HOL(Head-of-Line) 블로킹 문제를 완화할 수 있는 잠재력을 가졌지만, 인프라 전반의 변경을 요구하며 기존 네트워크 장비(미들박스)와의 호환성 문제를 야기할 수 있다.

이러한 문제들을 해결하기 위해, 본 연구의 기반이 되는 'eMQTT-AC'는 eBPF를 이용해 커널 신호를 관측하고, 이를 바탕으로 발행자의 전송률을 조절하는 휴리스틱 기반 적응형 제어 시스템을 제안했다. 이 시스템은 애플리케이션이나 브로커 수정 없이 엣지 게이트웨이에서 동작하며 꼬리 지연 시간을 효과적으로 줄이는 성과를 보였다. 하지만 eMQTT-AC는 사전에 정의된 고정된 임계값(예: RTT가 70ms를 초과하면)에 의존하여 제어 결정을 내린다. 이러한 정적 규칙 기반의 휴리스틱은 특정 네트워크 환경에서는 효과적일 수 있으나, 패킷 손실률, 지터, 배경 트래픽 등이 수시로 변하는 동적이고 예측 불가능한 실제 엣지 환경에서는 최적의 성능을 보장하기 어렵다. 규칙 기반 휴리스틱은 새로운 시나리오에 잘 일반화되지 못하는 본질적인 한계를 가지며, 이는 보다 지능적이고 적응적인 제어 메커니즘의 필요성을 시사한다.

1.4 연구 목표 및 제안 방법: 강화학습 기반 제어

본 논문의 주된 연구 목표는 기존 휴리스틱 제어의 한계를 극복하고, 고도로 동적인 엣지 환경에서도 MQTT 꼬리 지연 시간을 효과적으로 최소화하는 지능형 제어 시스템, eMQTT-RL을 설계, 구현 및 평가하는 것이다.

이를 위해 본 연구는 다음과 같은 핵심 가설을 설정한다: 심층 강화학습(DRL) 에이전트는 eBPF를 통해 실시간으로 커널 신호를 직접 관측하고, 애플리케이션 수준의 지연 시간으로부터 피드백(보상)을 받음으로써, 사전에 프로그래밍된 휴리스틱 정책보다 더 효과적이고 강건한 제어

전략을 자율적으로 발견할 수 있다.

이 가설을 검증하기 위해, MQTT 꼬리 지연 시간 제어 문제를 마르코프 결정 과정(Markov Decision Process, MDP)으로 수학적으로 정형화한다. eBPF가 수집한 커널 신호(RTT, 재전송, 버퍼 점유율)를 '상태(State)'로, MQTT 발행자의 전송률 및 배치 크기 조정을 '행동(Action)'으로, 그리고 종단 간 지연 시간과 처리량을 기반으로 계산된 값을 '보상(Reward)'으로 정의한다. 이 MDP를 해결하기 위한 알고리즘으로, 연속적인 상태 공간과 이산적인 행동 공간을 효과적으로 다룰 수 있는 Proximal Policy Optimization(PPO)를 채택한다. DRL 에이전트는 환경과의 지속적인 상호작용을 통해 시행착오를 겪으며, 장기적으로 누적 보상을 최대화하는 최적의 제어 정책(Policy)을 학습하게 된다. 이는 정적 규칙에서 벗어나, 데이터 기반의 동적이고 상황에 맞는 최적의 의사결정을 내리는 패러다임으로의 전환을 의미한다.

1.5 논문의 구성

본 논문은 다음과 같이 구성된다. 제2장에서는 MQTT 프로토콜, eBPF 기술, 그리고 네트워크 제어 분야에서의 강화학습 적용 사례 등 관련 연구를 심도 있게 고찰한다. 제3장에서는 eMQTT-RL 시스템의 전체 아키텍처를 제시하고, 제어 문제를 MDP로 모델링하는 과정과 PPO 에이전트의 구체적인 설계 내용을 상세히 기술한다. 제4장에서는 실험 환경을 구축하고, 제안하는 eMQTT-RL 시스템을 기저 상태 및 기존 휴리스틱 제어

시스템과 비교하여 성능을 정량적으로 평가하고, 학습된 정책을 분석한다. 마지막으로 제5장에서는 연구 결과를 요약하고, 본 연구의 의의와 한계, 그리고 향후 연구 방향을 제시하며 결론을 맺는다.

2 관련연구

2.1 MQTT 프로토콜과 IoT 지연 문제

MQTT(Message Queuing Telemetry Transport)는 IoT 환경에서 널리 사용되는 경량 퍼블리시/구독(pub-sub) 기반 프로토콜이다. MQTT는 중앙 브로커를 통해 메시지를 중계하며, 센서 노드 등의 퍼블리셔가 특정 토픽(topic)에 메시지를 발행하면, 해당 토픽을 구독한 구독자(subscriber)들에게 브로커가 메시지를 전달한다. MQTT는 TCP 위에서 동작하며 QoS(Quality of Service) 수준(0, 1, 2)별로 전달 신뢰도를 조절할 수 있는데, 높은 QoS(예: 2)의 경우 다중 확인 메시지로 오버헤드와 지연이 증가하는 trade-off가 있다. IoT 시스템은 대역폭이 제한적이고 기기 자원이 한정되므로, MQTT 통신에서 네트워크 혼잡, 브로커 부하, 메시지 크기 등에 따른 지연이 문제시된다. 특히 브로커에 다수의 클라이언트가 접속하여 팬-아웃(fan-out) 규모가 큰 경우, 커널 네트워크 스택을 통한 잦은 컨텍스트 전환과 큐 대기로 엔드-투-엔드 지연이 수십 ms 이상 크게 증가할 수 있다. 또한 부하가 일정 임계치를 넘으면 지연이 일명 "하키-stick" 형태로 급격히 증가하며 시스템 처리량도 급격히 악화되는 현상이 관찰된다. 이러한 꼬리 지연은 IoT 제어 응용의 실시간성을 떨어뜨리고, 데이터 손실이나 응답 실패로 이어질 수 있어 반드시 해결이 필요하다. 지연의 주요 원인을 살펴보면, 네트워크 전송 지연(거리, 라우터 홉수), 큐잉 지연(브로커/네트워크 대기열 적체), 프로토콜 처리 지연(MQTT 패킷 파싱/직렬화) 등이

복합적으로 작용한다. IoT 기기 특성상 에너지 절약을 위한 슬립 모드나 wake-up 지연도 존재하여, 요청 도착 시점과 실제 처리 가능 시점 사이에 구조적인 지연이 발생할 수 있다. 예를 들어 센서 노드가 저전력 모드에서 깨어나는 시간, 이전 주기의 작업 완료 대기 등이 누적되어 프로토콜 초기화 지연을 야기하며, 이는 일반적인 대기열 지연 모델에서 간과되기 쉽다. 결과적으로 버퍼 오버플로우, 처리 지연 증가, 패킷 손실 등이 발생하고, 심한 경우 임계 이벤트 누락이나 시스템 안전성 저해로 이어진다.

2.2 꼬리 지연(tail latency)과 제어 필요성

꼬리 지연은 전체 요청 중 상위 몇 퍼센트의 응답 지연이 평균에 비해 매우 큰 현상을 가리키며, 일반적으로 p95, p99와 같은 상위 분위 지연으로 측정된다. 분산 시스템에서 꼬리 지연은 소수 요청의 지연이 전체 서비스 품질을 결정하기 때문에 중요하다. 꼬리 지연을 완화하려는 전통적 기법으로는 여분 자원 할당, 요청 중복 실행(hedging), 우선순위 제어 등이 있다. 예를 들어 구글 등에서는 동일 요청을 여러 서버에 보내 가장 빨리 응답 오는 결과를 취하는 헤지 요청으로 tail latency를 줄이는 시도를 했다. 그러나 이러한 방법은 추가 자원 소모나 네트워크 부하를 유발하며, MQTT와 같은 단일 브로커 시스템에는 적용하기 어렵다. 또 다른 접근은 큐 관리 정책을 개선하는 것이다. 네트워크 라우터 분야에서는 앞서 언급한 DropTail, RED, CoDel 등의 알고리즘이 지연을 줄이고 큐를 안정화하기 위해 사용되어 왔다. DropTail은 버퍼가 가득 차면 신규 패킷

을 버리는 단순 전략이며, RED는 조기 랜덤 드롭으로 혼잡을 예방하고, CoDel은 큐잉 시간이 임계치를 넘은 패킷을 버려 지연을 통제한다. IoT 환경에서도 LoRaWAN, NB-IoT 등에서 이러한 정책이 임베디드 형태로 쓰이고 있지만, 장치 내부 상태(예: 슬립 모드로 인한 처리 불능 시간)를 고려하지 않고, 고정된 규칙으로 동작하므로 이질적인 트래픽 패턴에 대응하기 어렵다. 한편 어댑티브 동시성 제어 기법으로, Netflix는 적응형 동시 접속 제한(Adaptive Concurrency Limit)을 적용하여 인스턴스 당 허용 요청수를 지연 기반으로 동적으로 조절하고 있다. 이 방법은 최소 지연 대비 현재 지연의 비율(gradient)을 이용해 큐잉 발생 여부를 감지하고, 지연이 증가하면 허용 동시 요청수를 줄이는 AIMD 유사 알고리즘을 적용한다. 그 결과 과도한 부하에서 추가 트래픽을 즉시 거부하여 지연 상승을 막고 서비스의 가용성을 높이는 효과를 거두었다. 그러나 이러한 기존 제어 기법들은 휴리스틱 규칙이나 고정 파라미터에 의존하는 경우가 많아, 시스템 동적 특성에 완벽히 대응하지 못한다는 한계가 있다. IoT 환경에서는 트래픽 부하, 기기 상태, 네트워크 컨디션이 수시로 변하므로, 보다 지능적이고 적응적인 제어 전략이 요구된다.

2.3 eBPF를 활용한 커널 신호 관측

eBPF(extended Berkeley Packet Filter)는 리눅스 커널(version 4.4 이상)에 내장된 고성능 샌드박스 실행 환경으로, 사용자 정의 코드를 커널에 안전하게 주입하여 동적으로 커널 동작을 추적/변경할 수 있다. eBPF는 원래 패킷 필터링을 위한 기술로 시작되었으나, 현재는 네트워킹, 보안, 성능

모니터링 등 광범위한 영역에서 사용되고 있다. eBPF 프로그램은 커널의 tracepoint, kprobe, socket, traffic control 등 다양한 지점에 연결되어 실행될 수 있으며, 커널 이벤트에 대한 콜백으로 동작하면서 실시간으로 데이터를 수집하거나 커널의 행동을 변경할 수 있다. 예를 들어 eBPF를 이용하면 TCP 소켓의 send/ack 이벤트를 추적하여 RTT(왕복 시간)를 직접 측정하거나, TCP 재전송 발생 함수를 후크하여 재전송률을 계측할 수 있다. 또한 패킷이 네트워크 스택에 들어온 시점과 나간 시점을 기록하여 큐잉 지연을 산출하거나, 특정 시스템 콜(write)의 실행 시간을 추적하여 IO 병목을 찾는 등 다양한 활용이 가능하다. 특히 eBPF는 추가적인 하드웨어 장치 없이 프로덕션 시스템에서 저오버헤드로 세부 성능 모니터링을 할 수 있어, Netflix, Facebook 등의 대규모 서비스에서도 활용된다. Netflix는 eBPF 기반 스케줄러 모니터링으로 노이즈 성상(Noisy Neighbor) 문제를 탐지하였고, Facebook은 eBPF를 활용해 커널 레이턴시 관측 툴을 개발하였다. 또한 eBPF를 통해 사용자 공간을 우회한 네트워크 fast-path를 구현하면 RPC 프레임워크의 응답 지연을 줄일 수 있음이 보고되었다. 이처럼 eBPF는 MQTT 브로커/클라이언트의 커널 네트워킹 계층에 적용하여, 실시간 신호 수집과 동적 패킷 제어를 구현하는 강력한 도구가 될 수 있다. 본 연구의 전신인 eMQTT-AC에서도 eBPF를 통해 MQTT 통신의 RTT, 버퍼 점유율 등의 신호를 추출하고 메시지 흐름을 제어하였다.

2.4 강화학습을 이용한 네트워크 제어 동향

강화학습(RL)은 에이전트가 환경과 상호작용하며 시행착오(trial-and-error)를 통해 최적의 행동 정책을 학습하는 기계학습 기법이다. 최근 딥러닝 기술과 결합된 심층 강화학습(Deep RL)이 네트워크 제어 분야에도 도입되어 주목받고 있다. 복잡한 네트워크 환경에서는 전통적인 모델 기반 제어가 어려운 반면, RL은 환경으로부터 직접 피드백을 받아가며 비선형 최적 제어 전략을 스스로 학습할 수 있다는 장점이 있다. 선행 연구들에서는 네트워크 혼잡 제어에 RL을 적용하여 성능 향상을 달성했다. 예를 들어, Aglamazlar 등은 심층 Q-네트워크(DQN) 기반 TCP 혼잡 제어 알고리즘을 NS-3 시뮬레이터 상에서 구현하여, 기존 TCP New Reno 대비 지연 12.51% 감소, 처리율 68.31% 증가의 개선을 보고하였다. 이 연구는 RL이 동적으로 변화하는 네트워크 상황에 적응하여 패킷 혼잡 윈도우(cWnd)를 최적화함으로써, 전통적 알고리즘보다 나은 성능을 보일 수 있음을 보여준다. 또 다른 연구로 Kovtun 등은 분산 에지-IoT 대기열 제어 문제에 DQN 기반 RL을 적용하여, 평균 지연을 17.26% 감소시키고 서비스 시간 변동과 피크 부하 후 큐 안정성을 개선하는 결과를 얻었다. 이 연구에서는 다중 요인 보상 함수를 통해 지연, 변동성, 에너지 효율을 동시에 고려하였고, RL 에이전트가 지역 큐 상태를 보고 서비스 지연(phase shift)를 동적으로 조절하는 방법을 제시하였다. 그 외에도 네트워크 자율 제어 (예: 라우팅, 스케줄링) 영역에서 정책 경사 방법, Actor-Critic 계열 알고리즘(PPO, DDPG 등)을 활용한 연구가 증가하고

있다. 이러한 배경은, MQTT tail 지연 제어에도 RL을 도입하면 사람이 일일이 파라미터 튜닝을 하지 않아도 환경에 최적화된 제어 정책을 얻을 수 있으리라는 동기가 된다.

3 기존 시스템: eMQTT-AC

이 장에서는 본 연구의 기반이 된 eMQTT-AC 시스템을 살펴본다. eMQTT-AC는 MQTT 환경에서 커널 레벨 신호를 활용하여 적응형 제어 (Adaptive Control)를 수행함으로써, 부하 변화 시 꼬리 지연을 완화하도록 설계된 시스템이다. 구체적으로 eMQTT-AC는 리눅스 커널 내 eBPF 프로그램들을 이용해 네트워크 상태 신호를 수집하고, 이를 바탕으로 메시지 게이팅(gating) 정책을 적용하여 MQTT 메시지 전송률을 조절한다.

3.1 시스템 아키텍처와 동작 개요

eMQTT-AC의 전체 구조는 모니터링 모듈, 제어 결정 모듈, 실행 모듈의 피드백 제어 루프로 개략화할 수 있다. 모니터링 모듈은 브로커와 클라이언트의 커널에 상주하며, eBPF를 통해 실시간 신호를 추출한다. 예컨대 클라이언트 측 송신 소켓의 RTT는 패킷 송신 시각과 ACK 수신 시각을 관찰하여 측정하고, 송신 버퍼 점유율은 커널 소켓 버퍼의 사용량을 추적하여 얻는다. 또한 TCP 재전송률은 TCP 계층의 재전송 이벤트 발생 횟수를 계산하여 산출한다. 이러한 값들은 일정 주기마다 유저 공간의 제어 모듈로 전달된다. 제어 결정 모듈(Adaptive Controller)은 수집된 신호를 바탕으로 현재 네트워크 부하 상태를 평가하고, 게이팅 제어 신호를 산출한다. eMQTT-AC에서는 비교적 단순한 휴리스틱/제어 이론에 기반한 정책을 사용하였는데, 개념적으로 이는 Netflix의 적응형 동시성 제한

과 유사한 AIMD 알고리즘 또는 PI 제어기로 이해할 수 있다. 예를 들어 RTT가 최저 지연(base RTT) 대비 일정 비율 이상 증가하면 혼잡 발생으로 간주하여 전송 게이트를 닫고(=송신 일시 정지), RTT가 안정화되면 게이트를 다시 여는 방식이다. eMQTT-AC의 gating은 이진적 ON/OFF 제어로 구현되었으며, ‘게이트를 연다’는 것은 클라이언트가 브로커로 메시지를 전송하도록 허용하는 상태, ‘게이트를 닫는다’는 것은 새로운 메시지 발행을 일시 지연시키는 것을 의미한다. 이때 게이트 오픈 시간 및 닫는 조건은 버퍼 점유율 임계치나 연속 재전송 발생 여부 등을 종합적으로 고려하여 결정된다. 이러한 정책은 Queue Occupancy ; Θ_1 & RTT ; Θ_2 일 때 게이트 오픈, 넘어서면 닫는 식의 임계치 기반 제어로 볼 수 있다. 또한 과도한 스위칭을 방지하기 위해 히스테리시스(hysteresis)나 최소/최대 오픈 시간 제한 등을 두어 안정성을 높였다. 실행 모듈은 제어 모듈의 결정에 따라 실제로 메시지 전송을 제어하는 부분이다. eMQTT-AC에서는 게이트 제어 신호를 클라이언트 MQTT 퍼블리셔에게 전달하여, 게이트가 닫힌 동안은 퍼블리셔가 메시지 발행을 보류하도록 하였다. 구현 측면에서 제어 신호는 사용자 공간 프로세스 간 IPC나 브로커가 퍼블리셔에게 보내는 특수 제어 패킷 형태로 전송될 수 있다. 한 가지 구현 방식으로, 브로커가 클라이언트로 TEMP PAUSE 명령을 MQTT 확장 프로토콜 형태로 보내면 클라이언트 라이브러리가 수신하여 지정된 시간 동안 publish를 중단하는 식이다. 또는, eBPF를 활용해 소켓 레벨에서 패킷을 필터링하여 임의로 딜레이를 주는 방법도 가능하다. eMQTT-AC 프로토타입에서는 전자의 애플리케이션 계층 신호 전달 방식을 채택하였다. 게이트가 다시 열리면, 보류되었던 메시지들을 순차적으로 전송 재개한다. 이 과정에서 보류 시간이 너무 길어 타임아웃되거나 버퍼 오

버플로우가 발생하지 않도록, 게이트 닫기 최대 지속 시간이나 보류 버퍼 크기를 제한하였다.

3.2 커널 신호 기반 gating 정책의 효과와 한계

eMQTT-AC의 접근은 운영체제 커널의 세부 신호(RTT, 재전송 등)를 노출시켜 제어에 활용함으로써, 단순히 애플리케이션 레벨 지표(예: 응답 시간 평균)만으로는 감지하기 어려운 혼잡 징후를 빠르게 포착할 수 있다는 강점이 있다. 예를 들어 순수 MQTT 레벨에서는 알 수 없는 패킷 재전송 발생률은 네트워크 혼잡도의 직접적인 신호이며, eMQTT-AC는 이 값을 모니터링하여 재전송률 급증 시 즉각 게이트를 닫아 추가 부하를 억제하였다. 또한 RTT는 네트워크 큐잉 지연의 대표적인 지표로, 최적 RTT 대비 RTT가 일정 비율 이상 늘어나면 backlog가 커졌음을 의미한다. eMQTT-AC는 이러한 RTT 변화를 활용하여 대기열이 포화되기 전에 미리 전송을 억제함으로써 꼬리 지연이 악화되는 것을 방지했다. 실제 실험에서 eMQTT-AC는 burst 부하 상황에서 baseline (무제어) 대비 p99 지연을 상당 폭 줄이고, 지연 분산을 낮추는 결과를 보였다. 이는 gating을 통해 과도한 동시 요청을 적절히 제한함으로써 브로커 큐의 체류 시간을 줄인 데 따른 효과이다. 그러나 eMQTT-AC의 한계도 존재한다. 첫째, 정책 튜닝을 수동으로 해야 하므로 환경 변화에 보편적으로 최적화되기 어렵다. 예를 들어 임계치 Θ_1 , Θ_2 의 값은 네트워크 대역폭, 디바이스 성능 등에 따라 달라져야 하는데, 이를 사전에 모두 반영하기 어렵다. 둘째, gating과 같은 이진 제어는 상황에 따라 과도한 보수적 제어가 될 수 있다.

부하가 일시적으로 높아졌을 때 RL과 같은 기법은 점진적으로 전송률을 낮추는 세밀한 조정이 가능한 반면, eMQTT-AC는 문턱값 초과 시 전송을 완전히 멈추므로 일부 성능 손실로 이어질 수 있다. 셋째, eMQTT-AC는 복수의 상태 신호를 사용하지만, 이들을 종합하여 최적 결정을 내리는 로직은 개발자가 사전에 정의한 함수를 따른다. 다양한 신호 간 상관관계나 비선형 효과를 충분히 활용하지 못하며, 사전에 고려하지 못한 패턴(예: RTT는 낮지만 재전송이 높은 비정상 상황)에 대처하기 어려울 수 있다. 이러한 한계를 보완하기 위해 본 연구에서는 심층 강화학습을 통해 경험 기반 최적 정책을 학습하도록 하였다. 다음 장에서는 eMQTT-AC를 발전시킨 eMQTT-RL 시스템의 설계 내용을 자세히 다룬다.

4 강화학습 기반 확장: eMQTT-RL

이 장에서는 제안하는 eMQTT-RL 시스템의 내부 구성과 설계 원리를 설명한다. eMQTT-RL은 앞서 설명한 eMQTT-AC의 구조에 강화학습 에이전트를 통합한 형태로, 사람이 정한 규칙 대신 에이전트가 환경과의 상호작용을 통해 최적의 꼬리 지연 제어 정책을 학습하도록 한다. 주요 구성 요소로 상태(State) 공간, 행동(Action) 공간, 보상(Reward) 함수, 안전 장치(Shield), 학습 환경 및 알고리즘에 대해 순서대로 서술한다.

4.1 상태 공간 정의

eMQTT-RL 에이전트의 상태 (S_t)는 현재 MQTT 네트워크와 시스템의 상황을 나타내는 여러 커널 신호 피쳐들로 구성된다. 강화학습의 성능은 적절한 상태 표현에 크게 좌우되므로, 꼬리 지연과 밀접한 관련이 있는 신호들을 선정하였다. 주요 상태 변수는 다음과 같다:

- **RTT(Round Trip Time):** MQTT 패킷(예: PUBLISH 패킷)에 대한 왕복 시간 지연. 직전 기간 동안 측정된 평균 RTT나 90% RTT 값을 사용한다. RTT는 네트워크 혼잡으로 인한 큐잉 지연을 반영하는 핵심 신호이다. 값이 높아지면 네트워크 혹은 브로커 대기열에 지연이 누적되고 있음을 의미한다.
- **송신 버퍼 점유율:** 클라이언트 측 TCP 송신 버퍼의 현재 사용률 (0 100%). 이는 퍼블리셔가 보내려는 데이터가 커널 버퍼에 쌓여 있는 정도를 나타낸다. 버퍼 점유가 높으면 발행 속도가 네트워크 전송 속도를 초과하고 있어, 곧 패킷 지연이나 드롭이 발생할 가능성이 있다.
- **패킷 재전송률:** 단위 시간당 발생한 TCP 재전송 패킷 수 비율. 재전송은 패킷 손실이나 심각한 지연으로 ACK를 못 받은 경우 발생하므로, 네트워크 상태 악화를 나타내는 중요한 지표이다. 재전송률이 상승하면 링크 혼잡 혹은 신호 간섭 등이 증가했다고 볼 수 있다.
- **MQTT 대기열 길이:** 브로커 내부의 해당 클라이언트/토픽에 대한

대기 중인 메시지 수. 브로커가 처리해야 할 큐 길이가 길수록 서비스 지연이 증가할 가능성이 높다. (이 값은 브로커에서 eBPF로 노출되기 어렵다면, 클라이언트 입장에서 추정하거나 주기적 통계로 수신 가능하다고 가정한다.)

- **최근 p95 지연:** 매우 최근(예: 직전 1분)의 MQTT 메시지 전달 완료 시간 중 95번째 퍼센티지 지연 값. 이는 에이전트에게 현재 꼬리 지연 수준을 알려주는 지표로 사용된다. 만약 측정 창(window)이 충분히 짧다면 마치 실시간 꼬리 지표처럼 활용 가능하다. p95 지연이 높다면 곧 p99도 높아질 가능성이 있으므로 조기 대응이 필요하다.
- **기타:** 이 외에도 시스템 여유 CPU, 메모리 사용률, 현재 게이트 열림/닫힘 상태 등도 상태에 포함될 수 있다. 다만 본 연구에서는 핵심 네트워크 지표 위주로 상태를 설계하여 차원의 저주를 막고 학습 효율을 높이고자 했다.

상태 벡터(S_t)는 위 신호들을 정규화(예: 0-1 스케일)하여 구성하며, 시간 t 에 측정된 최신값뿐 아니라 이동 평균이나 과거 L 개의 관측치를 포함시켜 추세 정보를 반영할 수도 있다. 예를 들어 RTT의 최근 추세가 계속 상승 중인지 여부는 단순 현재값보다도 중요한 정보일 수 있어, 최근 5개 RTT 측정치들을 함께 상태로 제공하면 에이전트가 증가 추세를 인식할 수 있다. 이러한 상태 설계는 모두 꼬리 지연의 주요 원인인 혼잡과 과부하 상황을 에이전트가 파악할 수 있도록 하는 데 목표가 있다.

4.2 행동(Action) 정의

에이전트가 출력하는 행동 (A_t)은 MQTT 메시지 흐름 제어를 위한 조정 명령이다. eMQTT-RL의 행동 공간은 연속적인 제어 파라미터 2가지로 구성되며, 이는 (전송 속도 조정값 $\Delta rate$, 메시지 배치 크기 $batch_size$)이다. 각각의 의미는 다음과 같다:

- **$\Delta rate$ (전송 속도 증감율):** 현재 메시지 발행 속도를 증가 또는 감소시키는 값이다. 예를 들어 t 에서 $\Delta rate_t$ 가 -0.1이라면, 현재 퍼블리셔 전송율을 10% 줄이라는 의미이다. 이 값은 연속 액션으로 $[-R_{max}, +R_{max}]$ 범위를 가지며 (본 연구에서는 $R_{max}=0.2$ 로 설정, 한 번 행동에 최대 $\pm 20\%$ 조정), 에이전트는 이를 통해 세밀한 속도 제어를 수행한다. 양수면 전송 가속 (게이트 완화), 음수면 전송 감속 (게이트 조임)에 해당한다. $\Delta rate = -1.0$ 이면 전송을 완전히 중단하는 extreme도 선택 가능하다. 이를 통해 기존 eMQTT-AC의 on/off 이분법을 넘어, 상황에 따라 부분적인 트래픽 저감도 구현할 수 있다.
- **Batch size (배치 크기):** 메시지를 뭉쳐서 보낼 묶음 크기를 결정한다. 이는 한 번 게이트를 열었을 때 얼마나 많은 메시지를 연속 발행할지 혹은 브로커가 동시에 처리하는 메시지 뭉치 크기를 의미한다. 예를 들어 $batch_size = 5$ 이면 게이트가 열릴 때 5개의 MQTT 메시지를 빠르게 연속 전송하고 다시 게이트를 닫는 식의 동작이 된다. Batch size를 1로 하면 매 메시지마다 gating 체크를 하고, 크

게 잡으면 다소 burst 전송을 허용한다. 이 매개변수는 burstiness를 제어함으로써 지연 분포에 영향을 줄 수 있다. 행동 공간에서는 $batch_size$ 를 1부터 B_{max} 사이의 이산 값으로 취급하거나, 연속값을 받을림하여 사용한다. 본 연구에선 1 10 범위로 두었다.

행동 (A_t)은 주기적으로 (예: 매 100ms 또는 1초) 산출되며, 해당 주기 동안 퍼블리셔의 전송 정책을 결정한다. 구체적으로 퍼블리셔는 $\Delta rate_t$ 에 따라 직전 대비 전송 속도를 조정하고, $batch_size_t$ 만큼 메시지를 보낸 후 만약 $batch_size_t$ 보다 대기열에 메시지가 더 있다면 게이트를 잠시 닫았다가 다음 주기에 다시 열린다. $\Delta rate_t$ 를 전송 간격 조절에 해석하면, 에이전트 행동은 AIMD의 증가/감소 스텝을 매 순간 학습으로 정하는 것과 유사하다. Batch size ($batch_size_t$)는 $\Delta rate_t$ 와 연관되어 동작하며, $\Delta rate_t$ 가 음수로 큰 폭일 때 (즉 전송 감속 필요) $batch_size_t$ 도 작게 하여 한꺼번에 많이 안 보내게 할 수 있고, 반대로 $\Delta rate_t$ 이 양수일 때 $batch_size_t$ 도 키워서 전송을 적극적으로 하는 식의 정책을 에이전트가 스스로 찾게 된다. 행동 공간의 설계 철학은, 전송률(rate)과 전송 패턴(batch)을 모두 제어함으로써 지연에 영향을 주는 양적/질적 측면을 모두 다루자는 것이다. 전송률 조절은 평균 처리량과 평균 지연을 좌우하고, 배치 크기 조절은 지연 분산과 꼬리 부분(버스트로 인한 일시 부하)을 좌우한다. 에이전트는 두 변수를 함께 조절하여, 예컨대 약간의 버스트를 허용해도 꼬리 지연이 목표 이내이면 throughput을 높이고, 반면 tail 지연이 위험해지면 전송률을 낮추되 작은 배치로 보내 꼬리를 안정화시키는 식의 정책 조합을 학습할 수 있다.

4.3 보상 함수 설계

강화학습 에이전트는 보상 함수 $R(s, a)$ 를 극대화하는 방향으로 정책을 학습한다. 본 연구의 목표는 꼬리 지연을 줄이면서도 서비스 처리량을 유지하는 것이다. 이를 위해 보상 함수를 지연 관련 페널티와 전송 성능 관련 보상을 함께 고려하는 다목적 함수로 설계하였다. 우선 지연 페널티는 최근 측정된 p99 지연(또는 p95)을 기반으로 부여한다. 예를 들어 p99가 기준 임계값 T_{99} (예: 100ms)보다 크면 초과량에 비례하여 큰 음의 보상을 주고, 낮으면 0 또는 약한 페널티만 준다. 수식의 예로, $R_{lat} = -\max(0, \frac{p99 - T_{99}}{T_{99}})$ 처럼 p99가 기준보다 몇 배 높은지에 따라 -1 ~ -N까지 페널티를 줄 수 있다. 또한 지연 분포의 변동성도 고려하여 p99와 p50(중앙값) 차이 혹은 p95와 p99 격차 등에 대해 추가 페널티를 두었다. 이는 에이전트가 평균 지연만 낮추고 tail은 여전히 큰 상황을 피하고, 전체 분포를 골고루 낮추도록 유도하기 위함이다. 다음으로 처리량/효율 보상은 에이전트가 지연만 신경 쓰느라 과도하게 전송을 억제하는 것을 방지하기 위함이다. 극단적으로는 전송을 아예 안 하면 지연은 0으로 수렴하겠지만 서비스가 불가능해진다. 따라서 일정 목표 처리량 T_{put} (예: 90% 이상의 패킷 전달률)을 만족하지 못하면 페널티를 주고, 달성하면 약간의 양의 보상을 준다. 예컨대 초당 처리 메시지 수가 목표의 $x\%$ 일 때 $R_{thr} = +\alpha \cdot \min(x, 1)$ 와 같이 0 ~ α 사이 보상을 설정하였다. 또한 패킷 드롭이나 재전송이 발생하면 이는 네트워크 효율 저하로 간주하여 별도 페널티를 추가하였다. 이렇게 하면 에이전트가 무조건 전송을 멈추는 방향보다는, 적정 수준의 전송을 유지하면서 지연을 줄이는 균형 정책을

찾도록 유도된다. 마지막으로 안정성 항목으로서, 행동 변화가 지나치게 크거나 빈번할 경우 (예: 연속 스텝에서 Δr 변화량이 크면) 작은 페널티를 부여하여 정책의 연속성을 확보했다. 이는 학습 과정에서 노이즈를 줄이고, 실제 시스템에서 너무 출렁이는 제어를 하지 않도록 하는 효과가 있다. 총 보상 R_t 는 위 요소를 합산한 형태로, $R_t = w_1 \cdot R_{lat}(p_{99}, p_{95}) + w_2 \cdot R_{thr}(\text{throughput, drop}) + w_3 \cdot R_{stable}(\text{action stability})$ 로 정의되었다. 가중치 w_i 는 실험을 통해 조정했으며, 기본적으로 지연 항목에 가장 큰 비중을 두었다($w_1 = 1.0, w_2 = 0.3, w_3 = 0.1$ 등). 보상 함수를 이렇게 구성함으로써, 에이전트는 tail 지연을 줄이는 것을 최우선 목표로 삼으면서도 극단적 트레이드오프 (처리량 포기)를 회피하는 방향으로 학습하게 된다. 실제로 초기 실험에서 R_{thr} 항목이 없을 때 에이전트는 일부 에피소드에서 전송률을 0에 가깝게 떨어뜨리는 행위를 보였으나, 이후 처리량 보상을 추가하자 이런 행동이 사라지고 균형 잡힌 정책으로 수렴하였다.

4.4 안전 제약 조건 (셴드) 설계

강화학습 정책은 학습 초기에 또는 탐험 단계에서 비합리적이거나 위험한 행동을 출력할 가능성이 있다. 실시간 네트워크 제어에서는 이러한 잘못된 행동이 시스템 장애를 유발할 수 있으므로, 안전 장치(safety shield)를 마련하였다. eMQTT-RL의 셴드는 사전에 정의된 제약 조건을 위반하는 행동을 탐지하여, 이를 수정 또는 취소하는 역할을 한다. 주요 안전 제약은 다음과 같다:

- **최소 전송률 유지:** 에이전트가 Δr 값을 연속으로 크게 음수로 줄여

전송을 사실상 정지시키려 할 경우, 일정 하한선($rate_{min}$) 이하로 전송률이 떨어지지 않도록 강제한다. 예를 들어 현재 전송률이 100 msg/s인데 $\Delta r = -0.5$ 가 두 번 연속 나오면 25 msg/s로 감소시킬 의도일 수 있으나, $rate_{min} = 50$ msg/s로 설정하면 50 미만으로는 줄이지 않는다. 이를 통해 과도한 서비스 거부를 방지한다.

- **최대 전송률 제한:** 반대로 에이전트가 과도하게 전송률을 높이는 것도 위험하다. Δr 가 연속 양수로 커져 현재 네트워크 용량을 넘는 속도를 요구하면 패킷 손실이 폭증할 수 있다. 따라서 전송률 상한 $rate_{max}$ (예: 초기 용량의 120%)을 정하고 이를 넘는 행동은 잘라낸다.
- **급격한 행동 변화 금지:** 한 스텝에서 다음 스텝으로 Δr 가 큰 부호 변화(예: +0.5에서 -0.5)를 보이거나 `batch_size`를 극단적으로 바꾸는 것은 불안정성을 야기한다. 쉘드는 이러한 변화가 감지되면 해당 행동을 이전 스텝과 중간값 정도로 완화하여 적용한다. 이로써 RL 탐험에 의해 정책이 출렁이는 것을 막는다.
- **Queue 임계 초과 방지:** 브로커나 클라이언트 버퍼/큐 길이가 절대 임계값(예: 1000건)을 넘으면, 이때는 강제로 게이트를 닫는 등 비상 제어를 가한다. RL 행동과 무관하게 시스템을 보호하기 위한 장치다.

쉘드 메커니즘은 위 조건들을 if-then 규칙으로 구현하였다. 에이전트가 행동 a_t 를 내리면, 먼저 시뮬레이션 상에서 그 행동이 다음 상태에 끼칠 영향을 예측하거나(간단히 현재 큐와 속도로 추론) 조건 위반 여부를

검사한다. 만약 위반하면 해당 행동을 사전 정의된 안전한 행동으로 치환한다. 예를 들어 “큐 임계 초과 방지” 규칙에 의해 현재 대기열이 이미 한계치 근처이면 a_t 를 강제로 ($\Delta r = -1.0, b = 1$) 즉 즉각적인 전송 일시정지로 바꿀 수 있다. 이러한 쉴드 작동은 학습 초기에는 제법 빈번했으나, 학습이 진행됨에 따라 에이전트가 안전 영역 내의 행동만으로도 보상을 높일 수 있음을 인지하면서 점차 줄어들었다. 실제로 최종 학습된 정책은 쉴드 개입 없이도 대부분 안전 제약을 준수하는 행동을 취했다. 이는 쉴드가 단순히 안전을 확보할 뿐만 아니라, 안전한 정책 공간 내에서 학습을 가속하는 효과도 있음을 시사한다.

4.5 학습 환경 구성

에이전트 훈련을 위해 강화학습 환경을 구현하였다. 환경은 OpenAI Gym 인터페이스를 따르며, 에이전트(액터)는 앞서 정의한 상태를 관찰하고 행동을 내리면, 환경이 그에 따른 다음 상태와 보상을 반환한다. 우리의 MQTT 제어 문제에서는 실제 물리 시스템이 존재하므로 이상적으로 현실 환경에서 직접 학습하는 것이 좋겠지만, 안전 및 시간 상의 제약으로 인해 시뮬레이션 환경을 구축하여 학습을 진행하였다. 네트워크 시뮬레이터로는 NS-3을 이용하여 MQTT 통신을 모사하였다. NS-3 내에 간단한 MQTT 브로커와 클라이언트 모델을 구현하고, 해당 시뮬레이터를 Gym 환경의 step 함수에서 호출하였다. 매 스텝은 100ms의 시뮬레이션 시간에 대응되며, 에이전트 행동이 적용되면 이 기간 동안 퍼블리셔의 전송 간격, 배치 크기 등에 변화가

주어진다. 시뮬레이션은 Poisson 분포의 센서 데이터 생성, 지정된 네트워크 대역폭과 지연, 랜덤 패킷 손실 등의 조건을 반영하였다. 환경은 `step()` 수행 후 해당 100ms 구간의 평균 RTT, 재전송 횟수, 버퍼 점유율 변화, p95 지연 등을 계산하여 다음 상태로 반환한다. 그리고 보상은 앞서 정의한 식으로 이 구간의 통계치를 이용해 계산한다. 이러한 방식을 통해 현실 시스템의 복잡한 동작을 비교적 정확히 재현하면서도, 안전하게 에이전트를 훈련시킬 수 있었다. 시뮬레이터 파라미터는 실제 eMQTT-AC 실험 환경과 유사하게 설정하였다. 예를 들어 왕복 기본 지연은 약 20ms, 유효 대역폭 5Mbps, MQTT 메시지 크기 200바이트, 클라이언트 15개 등이었다. 트래픽 패턴은 burst 부하(예: 1초 간격으로 갑작스런 이벤트 몰림)와 평균 부하 변화(예: 1분 주기의 사인파 형태로 증감)를 모두 시나리오로 포함시켜, 에이전트가 다양한 상황을 겪고 학습하도록 했다. 또한 간헐적으로 패킷 손실(1%)과 지터(jitter)를 삽입하여, 실제 무선 환경의 불확실성도 반영하였다.

4.6 알고리즘 선정 및 구현

강화학습 알고리즘으로는 Proximal Policy Optimization (PPO)를 채택하였다. PPO는 액터-크리틱 계열 방법으로, 현재 정책과 새 정책의 차이를 클리핑(clipping)하여 크게 벌어지지 않도록 제한함으로써 학습 안정성을 높인 알고리즘이다. 우리 문제의 행동 공간이 연속적(특히 $\Delta rate$)이고 상태 공간이 다차원인 점을 고려할 때, PPO는 DDPG, SAC 등의 다른 방법에 비해 구현이 용이하고 비교적 적은 튜닝으로 안정적인 성능을 보

여주었다. 실제로 초기에는 DQN을 이산화된 행동 공간($\Delta rate$ 를 몇 단계로 이산화 등)으로 시도해보았으나, 동시 제어 변수가 2개이고 값 범위가 넓어 학습이 잘 되지 않았다. PPO는 Gaussian 분포 정책으로 연속 행동을 샘플링하고, KL divergence 기반으로 업데이트를 제한하므로 우리의 문제에 적합했다. 정책 신경망 구조는 비교적 간단하게 설계하였다. 입력층은 상태 벡터(차원 약 510)를 받아 2개 은닉층(각 64노드, ReLU 활성화)을 거쳐, 출력층에서 액터망은 2차원($\Delta rate, batch_size$) 평균을, 크리틱망은 스칼라 상태값을 출력한다. $\Delta rate$ 에 대해서는 tanh 활성화를 적용하여 -1 +1 범위로 자연스럽게 제한되도록 했고, $batch_size$ 는 110 범위의 이산값 중 하나를 선택하도록 softmax 10개 출력으로 구현했다. (사실상 PPO를 연속-이산 복합 출력으로 쓴 셈인데, stable-baselines 코드를 약간 수정하여 사용하였다.) 학습 하이퍼파라미터로, time-step 200,000까지 에피소드를 반복하며 Adam 옵티마이저(학습률 $3e-4$)로 업데이트했다. 1에피소드는 시뮬레이션 시간 60초로 정의하여 다양한 부하 패턴을 다 포함하도록 했고, $\gamma = 0.99$ 의 할인율, $\lambda = 0.95$ 의 GAE 파라미터를 사용했다. 약 3,000 에피소드 정도 학습한 후 정책이 수렴하는 것을 확인하였다. 훈련은 GPU가 없는 CPU 환경에서도 1시간 내 수렴하도록 경량화하였다. 이는 하나의 학습 단계가 짧은 시뮬레이션(0.1초)에 해당하여 매우 빠르게 진행되기 때문이다. 최종 학습된 PPO 정책은 관찰되지 않은 새로운 트래픽 시나리오에 대해서도 비교적 강인하게 꼬리 지연을 통제하는 결과를 보였다.

4.7 로그 기반 오프라인 학습 및 초기화

학습 효율과 성능 향상을 위해 로그 기반 오프라인 궤적 학습 기법을 병행하였다. 이는 eMQTT-AC 시스템이나 무제어 시스템을 실행하여 얻은 운영 로그(상태-행동-지연 결과 시퀀스)를 활용하여, 초기 정책을 사전 학습(pre-training)하거나, 학습 데이터로 리플레이하는 방법이다. 우리는 eMQTT-AC로 다양한 부하 시나리오를 실험하여 상태, 게이트 on/off 결정, 그 결과 지연이 어떻게 변했는지 로그를 수집하였다. 이 로그를 통해 행동에 대한 성능 사전 지식을 추출할 수 있다. 예를 들어 eMQTT-AC가 p99 지연이 높아지면 게이트를 닫는 행동을 했고 그 결과 지연이 낮아졌다면, 이는 ”높은 지연시 전송 억제”가 효과 있음을 나타낸다. 이를 강화학습에 반영하기 위해 Behavior Cloning 기법을 적용, 로그의 (상태→행동) 쌍들을 이용해 정책 네트워크를 감독학습으로 초기 학습시켰다. 구체적으로 1000여 개 로그 샘플을 모아 $\Delta rate$ 출력을 eMQTT-AC의 결정(on/off를 ± 0.5 정도 continuous로 매핑)으로 회귀 학습시켰다. 이렇게 초기화된 정책으로 RL 학습을 시작하니, 무작정 0부터 시작할 때보다 수렴 속도가 약 30% 빨라졌다. 또한 오프라인 RL 관점에서, 로그 데이터를 리플레이 버퍼에 넣고 중요도 샘플링하여 학습 시 일정 확률로 과거 궤적을 경험하도록 했다. 이를 통해 학습 중에 탐색하지 못한 상태 영역에 대한 간접 정보를 보강하였다. 다만, eMQTT-AC 로그는 최적 정책이 아니므로 이를 전적으로 모방하지는 않고, 경험 리플레이의 하나로만 사용하였다. 이러한 혼합 기법은 에이전트가 탐험 단계에서 겪는 성능 저하를 줄여 안전하게 학습하도록 도와주었다. 결과적으로 본 연구의 에이전트

는 비교적 짧은 학습으로도 양호한 정책에 도달할 수 있었다.

4.8 학습 결과 및 정책 특성

충분한 학습 후 최종 얻어진 eMQTT-RL 정책은 꼬리 지연 제어에 있어서 사람 설계 정책보다 향상된 성능을 나타냈다. 학습 과정에서 에피소드 누적 보상은 꾸준히 증가하여 약 500 에피소드 이후 거의 수렴 상태를 보였다. 아래 그림 1은 학습 진행 중 누적 보상의 변화 곡선을 나타낸 것으로, 초기에는 보상이 낮았으나 학습이 진행될수록 급격히 상승하여 안정화되는 모습을 확인할 수 있다. 이는 에이전트가 환경에 적응하여 점차 더 나은 tail 지연 제어 전략을 습득했음을 시사한다. 특히 약 200 에피소드 이후부터는 보상의 상승 속도가 둔화되는데, 이는 탐색 단계에서 벗어나 정책 수렴에 들어섰음을 보여준다.

Fig. 1: 강화학습 에이전트의 학습 곡선 (에피소드 진행에 따른 누적 보상). 초반에는 보상이 낮으나, 학습이 진행됨에 따라 점차 증가하여 약 500 에피소드 이후 수렴하였다.

학습된 정책의 동작을 분석한 결과, 다음과 같은 특성이 관찰되었다:

- 에이전트는 RTT 상승 추세를 조기에 감지하여 $\Delta rate$ 를 음수로 조정함으로써 부하를 선제적으로 완화했다. 이는 eMQTT-AC의 임계치 기반보다 미세하게 조정되어, 예를 들어 RTT가 완만히 상승할 때는 약간만 속도를 줄이고, 급격히 상승할 때는 과감히 줄이는 등 상황별 대응을 보였다.

- 재전송률이 특정 임계 이상 치솟으면 즉각 *batch_size*를 1로 낮추고 $\Delta rate$ 를 크게 음수로 함으로써, 일시적 전송 중지와 유사한 강한 제어를 걸었다. 이는 패킷 손실이 감지되면 곧바로 혼잡으로 인식하여 꼬리 지연 악화를 차단하려는 행동으로 해석된다.
- 부하가 줄어들 때는 $\Delta rate$ 를 서서히 양수로 높이며 전송률을 점진적 회복시켰다. 이때 *batch_size*도 점차 증가시켜 시스템이 안정화된 후 처리량을 최대화하려는 경향을 보였다. 이는 TCP의 AIMD 증가처럼 신중하게 용량을 탐색하는 모습으로, Netflix 적응 제어와 유사한 패턴이 자율적으로 나타났다.
- 에이전트는 eMQTT-AC와 달리 게이트 완전 닫는 상황을 최소화하였다. 대신 낮은 속도라도 지속 전송하며 모니터링을 이어가는 방향을 택했는데, 이는 완전 정지 후 재개할 때 발생하는 버스트를 방지하여 꼬리 지연을 한층 줄이는 효과를 냈다.
- 안전 쉴드의 제약 내에서 정책이 학습되었기 때문에, 최종 정책은 안전 임계선 근처의 행동을 거의 취하지 않음을 확인했다. 예컨대 전송률 하한 50 msg/s 제약이 있었는데, 정책은 최악의 상황에서도 60 70 정도로 유지하고 있었고, 상한도 넘지 않는 선에서 결정되었다. 이를 통해 학습된 에이전트가 쉴드 없이도 안전하게 운영 가능함을 보였다.

정리하면, eMQTT-RL 에이전트는 사람이 설계한 룰 기반을 넘어 여러 신호의 상관관계와 동적 패턴을 활용하여 세밀하고 상황맞춤형 제어를 학습했다. 다음 장에서는 이러한 학습 정책을 실제 시스템에 적용하여

얻은 성능 개선 결과를 상세히 평가한다.

5 평가 및 비교

본 장에서는 제안한 eMQTT-RL 기법을 실제 MQTT 시스템에 적용한 후 얻은 성능 평가 결과를 기존 기법들과 비교한다. 비교 대상은 (a) 무제어 Baseline, (b) 기존 eMQTT-AC, (c) 제안 eMQTT-RL의 세 가지이다. Baseline은 MQTT 프로토콜 표준 구현(예: Mosquitto 브로커+PAHO 클라이언트)을 사용하고 별도의 지연 제어를 하지 않은 경우이며, eMQTT-AC는 3장에서 설명한 커널 신호 기반 gating을 적용한 경우이다. 실험 환경은 앞서 시뮬레이션에서와 유사하게 구성한 테스트베드로, Mosquitto MQTT 브로커와 자체 개발한 퍼블리셔 클라이언트를 사용하였다. 클라이언트 3개가 1초당 100개의 센서 메시지를 발행하며 30초간 실행하고, 10초 시점에 2초 동안 3배 부하(burst)가 발생하는 시나리오로 테스트하였다. 각 실험 시나리오를 5회 반복 실행하여 평균을 취했다.

5.1 꼬리 지연 성능 비교

먼저 지연 분포에 대한 평가 결과를 살펴본다. 아래 그림 2는 Baseline, eMQTT-AC, eMQTT-RL 각각에 대한 상위 지연값(p95, p99)을 비교한 것이다.

그림 2에서 알 수 있듯, Baseline의 p99 지연은 약 300ms에 달하여

Fig. 2: p95 및 p99 꼬리 지연 비교 (Baseline vs eMQTT-AC vs eMQTT-RL). 제안하는 eMQTT-RL이 가장 낮은 꼬리 지연을 보이며, eMQTT-AC도 무제어 대비 개선됨을 알 수 있다.

세 기법 중 가장 높았다. 이는 부하 폭증 시 브로커의 큐잉 지연이 누적되고 패킷 재전송 등이 발생한 영향으로 보인다. eMQTT-AC를 적용한 경우 p99 지연이 약 200ms로 감소하여 Baseline 대비 약 33% 개선되었다. p95 지연 역시 Baseline의 120ms에서 90ms 수준으로 낮아졌다. 이는 gating 정책이 혼잡 시 일시적으로 트래픽을 억제함으로써 tail 부분의 응답 시간을 단축한 결과이다. 가장 우수한 성능을 보인 것은 eMQTT-RL로, p99 지연이 약 150ms까지 감소하여 Baseline 대비 50% 낮은 값을 기록하였다. p95 지연도 80ms 수준으로 세 기법 중 최저였다. eMQTT-RL은 eMQTT-AC와 비교해도 p99을 약 25% 더 줄였는데, 이는 RL 에이전트가 gating을 세밀하게 조절하여 필요 이상으로 큐가 커지지 않도록 한 덕분이다. 결과적으로 eMQTT-RL은 꼬리 지연 측면에서 가장 뛰어난 성능을 입증하였다. 세 기법의 지연 분포 곡선을 살펴보면, Baseline은 분포 꼬리가 두텁고 p99 이상 영역에 일부 초고지연(outliers)이 존재했다 (최대 500ms 수준). eMQTT-AC의 분포는 꼬리가 짧아졌으나 여전히 p99 이후 약간의 긴 꼬리가 있었다. 반면 eMQTT-RL은 분포가 가장 가파르게 떨어지며 p99 이후 거의 outlier가 없었다. 즉 RL 제어가 지연 분포의 긴 꼬리를 효과적으로 잘라냈음을 알 수 있다.

5.2 평균 지연 및 시스템 처리량

지연 평균값 측면에서는 세 기법 간 큰 차이는 없었다. Baseline의 평균 응답 지연이 약 45ms, eMQTT-AC가 50ms, eMQTT-RL이 48ms 정도였다. eMQTT-AC와 RL 모두 평균 지연이 소폭 증가했는데, 이는 꼬리 지연을 낮추기 위해 일부 요청을 지연시키는 trade-off 때문에 발생한 현상으로 분석된다. 예컨대 gating으로 트래픽을 제어하면 전체 응답 중 빠른 응답들도 약간 지연되어 나오므로 평균이 늘어날 수 있다. 그러나 이 정도 증가는 5~10% 이내로 미미한 수준이며, 꼬리 지연 감소로 인한 이득에 비하면 감내할 수 있다. 특히 IoT 제어계에서는 평균 지연보다는 일관된 응답 시간이 중요하므로, eMQTT-AC와 RL의 접근이 유용함을 보여준다. 한편 시스템 처리량(단위 시간당 처리 메시지 수)은 Baseline과 eMQTT-RL이 비슷했고, eMQTT-AC가 약간 낮았다. Baseline은 제어를 안 하기 때문에 가능한 한 많은 트래픽을 처리하려 하고, RL은 지연을 보존하면서도 처리량을 유지하도록 학습되었으므로 불필요한 억제를 하지 않았다. 반면 eMQTT-AC는 임계 초과 시 전송을 정지하기 때문에, burst 구간에서 일시적으로 처리가 중단되어 전체 처리량이 약 5% 감소하였다. RL은 burst 구간에서도 완전 중단 대신 낮은 속도로나마 지속 처리하여, 누적 처리량 손실이 거의 없었다. 이는 RL 정책이 지연과 처리량의 균형을 잘 잡았음을 의미한다. 다시 말해 RL은 tail latency를 줄이면서도 throughput penalty를 최소화했다.

5.3 동적 부하 대응 분석

부하 변화에 따른 세 기법의 동작을 시간축으로 살펴보면 흥미로운 차이가 나타난다. 실험에서 10초에 트래픽이 3배로 급증하고 12초에 원상태로 복귀하도록 설정하였는데, Baseline의 경우 10 12초 구간에 응답 지연 분포가 크게 넓어지고 p99 응답이 최고 400ms까지 치솟았다. eMQTT-AC는 10초 직후 gating이 발동하여 1초 가량 전송을 멈추면서 지연 폭주는 막았으나, 그 영향으로 10 11초 사이 메시지 처리가 지연되어 해당 구간의 p50도 상승했다. eMQTT-RL은 10초 진입과 동시에 $\Delta rate$ 를 단계적으로 낮춰 약 30% 전송량을 감소시켰고, *batch_size*도 1로 유지하여 큐 폭증을 방지하였다. 그 결과 p99 지연이 200ms 선에서 안정화되었고, 11초 경부터는 부하가 지속 높음에도 불구하고 응답 지연이 더 증가하지 않았다. 또한 12초 부하 감소 시 RL은 서서히 전송률을 올려 13초경 평상시 수준을 회복했다. eMQTT-AC의 경우 12초 부하가 줄어들자 게이트를 바로 열어버려 한꺼번에 밀린 메시지가 처리되며 12초 후반에 약간의 지연 스파이크가 나타났다. 이는 RL의 점진적 재개 vs AC의 즉시 재개 차이로 볼 수 있다. 결국 RL은 부하 변화 앞뒤로 과도한 진동 없이 부드럽게 제어된 반면, AC는 다소 널뛰기가 있었다.

5.4 추가 오버헤드 및 구현 복잡도

새로운 RL 기반 제어를 도입함에 따라 발생하는 추가 오버헤드와 구현 이슈도 평가하였다.

- **CPU 오버헤드:** eMQTT-RL은 주기적으로 신경망 추론을 수행해야 하므로 eMQTT-AC 대비 CPU 부하가 늘어난다. 측정 결과 Python 기반 구현에서 추론 시간은 한 번에 약 12ms 정도로 나타났으며, 이는 100ms 제어 주기 대비 12%에 불과하다. C++로 최적화하거나, TensorRT 등을 사용하면 이마저도 미미해질 것으로 보인다. 따라서 현재 스펙의 IoT 게이트웨이에서도 충분히 실시간 동작이 가능하다.
- **메모리 오버헤드:** 정책 신경망 파라미터 (수백 KB)와 최근 상태 버퍼 등으로 수 MB 수준의 메모리가 추가 소요된다. 브로커나 클라이언트 시스템에는 문제가 없는 크기다.
- **eBPF 프로그램 복잡도:** eMQTT-AC에서 eBPF로 측정하는 항목들은 동일하게 활용된다. RL 적용으로 eBPF 측이 더 복잡해지지는 않았다. 제어 신호를 전달하는 방식은 AC와 동일하게 구현했으며, RL에서는 gating이 on/off가 아니므로 세분화된 제어 명령(예: "전송 속도 0.8배로 조절") 형태로 프로토콜 확장을 하였다. 이 역시 큰 어려움 없이 구현 가능했다.
- **개발 난이도:** RL 도입으로 학습 환경 구성, 시뮬레이터 제작, 파라미터 튜닝 등 초기 개발 노력이 증가했다. 반면 일단 학습이 완료되고 나면 운영 단계에서는 정책만 탑재하면 되므로, 운영 파라미터 튜닝 부담은 크게 줄었다. eMQTT-AC는 환경 바뀔 때마다 임계값 등을 재조정해야 했지만 RL 정책은 다양한 상황을 포괄하도록 학습되었으므로 추가 손질 없이 잘 작동했다. 따라서 개발 초기 vs 운영 단계의 노력 균형상, RL 접근도 충분히 현실성 있다 판단된다.

다.

5.5 요약

평가 결과를 종합하면, eMQTT-RL 시스템은 꼬리 지연 제어 성능 면에서 기존 적응형 제어(eMQTT-AC)를 뛰어넘었고, 무제어 대비 큰 향상을 달성했다. 특히 p99 지연을 절반 수준으로 줄이고도 처리량 손실이 거의 없다는 점은, RL을 통해 지연-처리량 간 최적 균형점을 찾아냈음을 보여준다. eMQTT-AC 역시 baseline 대비 유의미한 개선을 제공하지만, RL은 이를 더욱 개선함과 동시에 자동화된 적응 능력을 갖추었다. 한편 RL 도입에 따른 오버헤드는 실시간 제어에 무리가 없는 수준이었으며, 이는 현대 IoT 게이트웨이나 서버에서 충분히 수용 가능하다. 이러한 결과는 커널 신호 기반 RL 제어라는 본 논문의 접근이 MQTT 뿐만 아니라 다른 네트워크 시스템의 tail latency 문제에도 효과적일 가능성을 시사한다. 다음 장에서는 본 연구의 결론을 요약하고, 남은 한계 및 향후 연구 방향을 논의한다.

6 결론 및 향후 연구

본 논문에서는 강화학습 기반의 eMQTT-RL 시스템을 제안하여, IoT MQTT 환경에서의 꼬리 지연(tail latency) 문제를 효과적으로 제어하고자 하였다. eMQTT-RL은 리눅스 커널의 eBPF 신호(RTT, 버퍼 점유율, 재전송률 등)를 실시간으로 활용하고, PPO 알고리즘으로

학습된 정책에 따라 MQTT 메시지 전송률과 배치 크기를 동적으로 조절함으로써 꼬리 지연을 낮추었다. 기존 적응형 제어 기법 (eMQTT-AC)과 비교하여, 제안 시스템은 상황 변화에 대한 적응성과 제어 정밀도 측면에서 우수함을 보였으며, 그 결과 p95, p99 지연을 현저히 감소시켰다. 실험 평가에서 eMQTT-RL은 baseline 대비 p99 지연을 50%까지 줄이면서 처리량은 동등하게 유지하는 성능을 입증하였다. 이는 강화학습을 통한 자동 최적화가 MQTT QoS 제어에 실효성이 있음을 보여준다. 향후 연구로는 다음과 같은 방향을 고려하고 있다:

- **온라인 학습 및 실시간 적응:** 본 연구에서는 사전 시뮬레이션 환경에서 정책을 학습하여 적용하는 오프라인 학습 접근을 취했다. 추후에는 실제 운영 환경에서 에이전트가 지속적으로 학습(online learning)하여, 장기간의 환경 변화나 트래픽 패턴 드리프트에도 스스로 적응하도록 하는 방안을 모색할 것이다. 이를 위해선 안전을 해치지 않는 범위에서 탐험을 허용하거나, 연속 학습(continuous learning) 프레임워크를 도입해야 한다.
- **다중 에이전트 및 분산 RL:** 한 브로커에 다수의 클라이언트가 연결된 경우, 각 클라이언트별 RL 에이전트를 둘 수도 있고 브로커 단위의 통합 에이전트를 둘 수도 있다. Multi-agent RL이나 분산 강화학습 기법을 적용하면, 여러 퍼블리셔 간의 상호작용까지 고려한 꼬리 지연 제어가 가능할 것이다. 예컨대 브로커가 에이전트로 동작하며 전체 큐를 관장하고, 개별 클라이언트 제어는 하위 에이전트들이 수행하는 계층적 RL 구조도 구상할 수 있다.

- **통합 자원 제어:** 꼬리 지연은 네트워크뿐 아니라 시스템의 CPU 부하, 디스크 IO 대기 등도 영향을 준다. 향후에는 RL 상태에 CPU 사용률, 스레드 풀 대기시간 등도 포함시키고, 행동으로 네트워크 외에 애플리케이션 레벨의 처리를 지연시키거나 우선순위 조정을 하는 등 종합적인 제어를 연구할 계획이다. 이를 통해 MQTT 브로커 내의 엔드투엔드 지연 경로 전체를 RL로 최적화하는 교차 계층 제어를 실현할 수 있을 것으로 기대한다.
- **이론적 보장:** RL 정책의 행위에 대해 지연 상한 보장이나 안정성 보장을 제공하는 것도 중요하다. 안전 강화학습이나 형식적 검증 기법을 활용하여, 학습된 정책이 주어진 지연 SLO(Service Level Objective)를 어기지 않음을 검증하거나, 만약 어길 경우 특정 조건 하에서만 어긴다는 확률적 보장 등을 연구할 필요가 있다. 이는 실제 산업 현장에 적용하기 위한 신뢰성 제고 측면에서 필수적인 과제이다.
- **일반화 및 타 프로토콜 적용:** 본 연구의 방법론을 MQTT 외의 다른 IoT 프로토콜(예: CoAP)이나 일반적인 TCP 응용으로 확장하는 것도 흥미로운 방향이다. eBPF 신호와 RL 기반 제어의 조합은 웹 서비스의 응답 지연 제어, 데이터센터 네트워크 혼잡 완화 등에도 활용될 수 있다. 이를 위해 각 도메인에 맞는 상태/행동 정의와 보상 함수 튜닝이 필요하겠지만, 기본 아이디어는 재사용 가능하다.

요약하면, 강화학습 + 커널 신호 기반 제어는 IoT 시대의 지연 문제 해결을 위한 강력한 도구임을 확인하였다. 본 연구를 통해 MQTT 시스템에서

효율적이고 자동화된 tail 지연 제어가 가능함을 보였으며, 이는 앞으로 더욱 지능화된 네트워크 제어 기술의 한 예시가 될 것이다. 향후 연구를 지속함으로써, 실시간성 요구가 높은 다양한 분산 시스템에서 스스로 학습하며 최적 성능을 유지하는 자율 제어를 실현할 수 있기를 기대한다.

APPENDICES

A

B